GIT & GITHUB

Complete Learning Guide

From first concepts to collaboration, undoing changes, and advanced workflows

Designed for: Beginners using Windows Command Prompt / PowerShell, with examples based on a Student Management API project in C#.

How to use this guide

- Read the concept before typing commands.
- Use a practice repository first. Do not experiment with destructive commands on important work.
- Commands beginning with # contain comments explaining the command below or beside them.
- Replace placeholder values such as COMMIT_ID and <repository-url> with your actual values.
- After each exercise, run git status and inspect the result.
- Git and GitHub are related but different: Git tracks history locally; GitHub hosts repositories and adds collaboration features.

Contents and 25-Lesson Roadmap

Lesson	Topic
1	Version control and Git fundamentals
2	Install Git and configure identity
3	Create a repository: git init
4	Working directory, staging, and commits
5	Inspect changes: status, diff, and log
6	Write good commits and use .gitignore
7	Branches and HEAD basics
8	GitHub repositories and remotes
9	Connect local repository to GitHub
10	Push, clone, and first remote workflow
11	Fetch, pull, and remote-tracking branches
12	Create, switch, rename, and delete branches
13	Feature-branch workflow
14	Merge basics and merge types
15	Merge strategies and merge verification
16	Resolve merge conflicts
17	Pull Requests: purpose and vocabulary
18	Create, review, and merge a Pull Request
19	Team collaboration, reviews, and branch protection
20	Forks, issues, and contribution workflow
21	Undoing changes: restore, revert, reset, amend
22	Stash: temporarily shelf unfinished work
23	Tags and releases
24	Rebase and history cleanup
25	Cherry-pick, reflog, troubleshooting, and safe workflow

The order is a learning roadmap. You can use the guide as a reference later rather than reading it only once.

PART 1 — FOUNDATIONS

Lesson 1 — Version Control and Git Fundamentals

Concept

Version control records how files change over time. Git is a distributed version-control system: each clone contains repository history, so many operations work locally.

Why it matters

Without version control, developers often create copies such as final.cs, final2.cs, final_really_final.cs. Git records meaningful snapshots and helps teams compare, collaborate, and recover.

Example scenario

While building a Student Management API, you add a student model, then a controller, then validation. Git lets you identify which change introduced a bug and return to a known state.

What to expect / verify

Git stores commits and relationships between them. It does not automatically save every keystroke; you decide when to create commits.

Common mistakes and cautions

- Git is not the same as GitHub. Git is the version-control tool; GitHub is a hosting and collaboration platform.

Remember: Think of Git as a detailed project timeline, not just an online backup.

Lesson 2 — Install Git and Configure Identity

Concept

Git needs an author name and email to label commits. This identity is metadata; it is not your GitHub password.

Why it matters

Correct identity makes it easier to identify who authored each commit.

Example scenario

You're setting up Git on a Windows laptop before starting your Student Management API repository.

Commands with comments

```
# Check that Git is installed and available in PATH
git --version

# Configure the author name for commits made by this user account
git config --global user.name "Your Name"

# Configure the email associated with your commits
git config --global user.email "you@example.com"

# Inspect the settings
git config --global --list
```

What to expect / verify

Git prints a version number and displays your configured name and email. Use an email you are comfortable associating with your commits.

Common mistakes and cautions

- Do not enter your GitHub password as the Git email. GitHub authentication is handled separately, often through a browser, credential manager, or SSH key.

Remember: Use --global for all repositories under your user account; omit it to configure only the current repository.

Lesson 3 — Create a Repository with git init

Concept

A repository is a project folder that Git tracks. `git init` creates the hidden `.git` directory that stores Git metadata and history.

Why it matters

Initialize a repository when you have a local project that is not already tracked by Git.

Example scenario

You created a folder for the Student Management API and want to start tracking its files.

Commands with comments

```
# Move to your project folder (Windows example)
cd C:\Projects\StudentManagementAPI

# Initialize Git in this folder
git init

# Check the repository state
git status
```

What to expect / verify

Git reports that a repository was initialized and shows the current branch and untracked files.

Common mistakes and cautions

- Do not run git init repeatedly in random parent folders. First check where you are with `pwd` in Git Bash or `cd` in Command Prompt and inspect `git status`.

Remember: `git init` starts tracking history; it does not automatically add files or create a commit.

Lesson 4 — Working Directory, Staging, and Commits

Concept

Git uses three key areas: working directory (your edits), staging area (changes selected for the next commit), and repository history (committed snapshots).

Why it matters

Staging lets you choose exactly what goes into a commit, rather than committing every modification in the folder.

Example scenario

You create `Student.cs` and `StudentController.cs`, but you want to commit only the model first.

Commands with comments

```
# See new and modified files
git status

# Stage only the student model
git add Student.cs

# Review what is staged
git diff --staged

# Save the staged snapshot in history
git commit -m "Add student model"
```

What to expect / verify

After the commit, the staged changes are recorded in history. Other unstaged changes remain outside that commit.

Common mistakes and cautions

- `git add .` stages many changes in the current folder and below. Review `git status` first if you do not intend to stage everything.
- A commit records staged changes, not every file automatically.

Remember: Small, focused commits are easier to understand and undo.

Lesson 5 — Inspect State: status, diff, and log

Concept

`git status` summarizes the working tree; `git diff` shows unstaged changes; `git diff --staged` shows staged changes; `git log` shows commit history.

Why it matters

Inspecting state before commands prevents accidental loss and makes Git behavior easier to predict.

Example scenario

You changed validation in `StudentService.cs` and want to check the exact difference before committing.

Commands with comments

```
# Summarize staged, unstaged, and untracked files
git status

# Show unstaged changes in tracked files
git diff

# Show staged changes compared with HEAD
git diff --staged

# Show recent commits in compact form
git log --oneline -5

# Show branches and commits as a graph
git log --oneline --graph --decorate --all
```

What to expect / verify

`git diff` can be empty while `git diff --staged` shows changes, because staged and unstaged changes are different.

Common mistakes and cautions

- New untracked files do not appear in ordinary `git diff` until staged; use `git status` to see them.
- Do not guess what a command will affect—inspect the state first.

Remember: Make `git status` a habit before and after significant Git operations.

Lesson 6 — Good Commit Messages and .gitignore

Concept

A commit message should summarize the intent of a change. `.gitignore` tells Git which untracked files it should normally ignore.

Why it matters

Build outputs, temporary files, logs, local secrets, and editor files usually should not be committed.

Example scenario

In a .NET project, generated `bin/` and `obj/` folders should generally not be committed. A real secret such as a production connection string must never be added to the repository.

Commands with comments

```
# Example .gitignore entries for a .NET project
# Build output
bin/
obj/

# IDE user-specific files
.vs/
*.user

# Local environment files (review your team's policy)
.env

# Stage the ignore file
git add .gitignore

# Commit the ignore rules
git commit -m "Add .NET gitignore rules"
```

What to expect / verify

Git ignores matching untracked files. `.gitignore` does not automatically stop tracking a file that was already committed.

Common mistakes and cautions

- Never commit passwords, API keys, private certificates, or production secrets. If a secret was committed, removing it in a later commit does not erase it from history; rotate the secret and follow a history-cleanup process if needed.
- Avoid vague messages such as `changes` or `update`.

Remember: Commit intent, not just file names. Ignore generated files before adding them.

PART 2 — GITHUB AND REMOTES

Lesson 7 — Branches and HEAD Basics

Concept

A branch is a movable name pointing to a commit. `HEAD` usually identifies the currently checked-out branch and therefore the commit you are working from.

Why it matters

Branches allow features and fixes to be developed separately from the stable line of work.

Example scenario

You want to add student search without mixing incomplete work into `main`.

Commands with comments

```
# List local branches; * marks the current branch
git branch

# Create a branch at the current commit (does not switch)
git branch feature/student-search

# Switch to an existing branch
git switch feature/student-search

# Or create and switch in one command
git switch -c feature/student-search
```

What to expect / verify

Both branches initially point to the same commit. Later commits on one branch move that branch pointer without automatically moving the other.

Common mistakes and cautions

- Uncommitted edits live in your working directory, not exclusively inside a branch. Git may prevent switching if changes would be overwritten.
- Prefer `git switch` for branch switching; use `git restore` for restoring files.

Remember: A branch is a pointer to history, not a separate physical copy of the whole project.

Lesson 8 — GitHub Repositories and Remotes

Concept

A remote is a named reference to another repository location. `origin` is the common default name for the remote you cloned from or added.

Why it matters

GitHub hosts a copy of a repository and provides web-based browsing, issues, pull requests, and collaboration.

Example scenario

You create an empty repository on GitHub to host the Student Management API.

Commands with comments

```
# List configured remotes and their URLs
git remote -v

# Add a remote if one is not configured
git remote add origin https://github.com/USERNAME/REPOSITORY.git

# Inspect remote names
git remote
```

What to expect / verify

The local repository can have zero, one, or multiple remotes. A remote name is just a shortcut to a URL.

Common mistakes and cautions

- If Git says `remote origin already exists`, inspect `git remote -v` instead of adding another origin. Use `git remote set-url origin <url>` only if you intentionally need to change it.

Remember: `origin` is a conventional name, not a special mandatory word.

Lesson 9 — Connect a Local Repository to GitHub

Concept

Connecting a local repository to GitHub lets you exchange commits with a remote repository.

Why it matters

Local history and GitHub history may start separately. You need to understand which one already contains commits before combining them.

Example scenario

You already have a local README and create a GitHub repository. If both sides have independent commits, Git may require you to integrate their histories before pushing.

Commands with comments

```
# Inspect current branch and working-tree state
git status
git branch

# Inspect the remote URL
git remote -v

# Add origin only if it does not already exist
git remote add origin https://github.com/USERNAME/REPOSITORY.git

# Rename the current branch to main if that is your intended convention
git branch -M main
```

What to expect / verify

Before the first push, verify the repository URL and whether the remote is empty or already has commits.

Common mistakes and cautions

- Do not blindly force-push to resolve a rejected push. If GitHub has a README or other commits that are absent locally, fetch and inspect the histories first.

Remember: Understand both histories before trying to combine them.

Lesson 10 — Push, Clone, and First Remote Workflow

Concept

`git push` uploads local commits; `git clone` creates a local copy of a remote repository including its history and remote configuration.

Why it matters

Push shares your commits. Clone is the usual starting point when joining an existing project on another machine.

Example scenario

You finish a README update and want it on GitHub, or a teammate wants to download the project.

Commands with comments

```
# Push the current branch and set its upstream tracking branch
git push -u origin main

# Clone a repository into a new local folder
git clone https://github.com/USERNAME/REPOSITORY.git

# Enter the cloned repository
cd REPOSITORY

# Inspect recent history
git log --oneline -5
```

What to expect / verify

After a successful push, the commit is available on GitHub. A clone includes the committed history, not other people's uncommitted working files.

Common mistakes and cautions

- Push can be rejected if the remote contains commits you do not have. Do not use force-push without understanding the remote history.
- Do not clone into an existing non-empty project folder unless you understand the consequences.

Remember: Push sends commits, not a live folder synchronization of every unsaved edit.

Lesson 11 — Fetch, Pull, and Remote-Tracking Branches

Concept

`git fetch` downloads remote objects and updates remote-tracking references such as `origin/main`, without integrating them into your current branch. `git pull` fetches and then integrates changes into the current branch.

Why it matters

Fetch lets you inspect incoming work before merging it. Pull is convenient when you are ready to update your current branch.

Example scenario

A teammate merged a fix into GitHub's `main`; you want to inspect it and then update your local `main`.

Commands with comments

```
# Download remote updates without integrating them
git fetch origin

# Compare commits in origin/main that local main does not contain
git log --oneline main..origin/main

# Switch to main before updating main
git switch main

# Fetch and integrate the remote main into local main
git pull origin main
```

What to expect / verify

`origin/main` is a remote-tracking reference, not the same thing as your local `main` branch.

Common mistakes and cautions

- `git pull origin main` integrates the remote main into whichever branch is currently checked out. Switch to main first if that is your goal.
- Pull can cause conflicts if histories have diverged.

Remember: Fetch to inspect; pull to fetch and integrate.

PART 3 — BRANCHING AND MERGING

Lesson 12 — Create, Switch, Rename, and Delete Branches

Concept

Branch commands manage branch pointers. `git switch` changes the checked-out branch; `git branch` lists, creates, renames, or deletes branch names.

Why it matters

Clear branch naming makes feature work and code review easier to follow.

Example scenario

You need a branch for registration, then a separate branch for a login bug fix.

Commands with comments

```
# Create and switch to a feature branch
git switch -c feature/student-registration

# List local branches
git branch

# Rename the current branch
git branch -m feature/registration

# Switch to main before deleting a finished local branch
git switch main

# Delete a local branch that has been merged
git branch -d feature/registration
```

What to expect / verify

Use `git branch -d` for a branch Git considers safely merged. `git branch -D` forces deletion and may discard the only branch name pointing to unmerged commits.

Common mistakes and cautions

- Do not delete a branch until you know whether its commits are merged or otherwise backed up.
- `git checkout` can also switch branches, but `git switch` communicates intent more clearly.

Remember: Use `git switch -c name` to create and switch in one step.

Lesson 13 — Feature-Branch Workflow

Concept

A feature-branch workflow keeps each feature or bug fix on its own branch, usually based on `main`.

Why it matters

It reduces accidental mixing of unrelated work and gives each change a focused review path.

Example scenario

You implement student search separately from student registration.

Commands with comments

```
# Start from an up-to-date main branch
git switch main
git pull origin main

# Create a feature branch
git switch -c feature/student-search

# Make code changes, then inspect them
git status
git diff

# Stage and commit the intended files
git add StudentController.cs StudentService.cs
git commit -m "Add student search"

# Publish the feature branch
git push -u origin feature/student-search
```

What to expect / verify

The feature branch is now available on GitHub and can be used for a Pull Request.

Common mistakes and cautions

- Keep commits focused. Do not commit build output or secrets.
- If main has advanced, update your branch using the team's merge or rebase policy.

Remember: Build a feature in isolation, then integrate it through review.

Lesson 14 — Merge Basics and Merge Types

Concept

`git merge BRANCH` integrates the named branch into the currently checked-out branch. The current branch is the destination.

Why it matters

Merge combines work from two lines of development.

Example scenario

You completed student search and want to integrate it into `main`.

Commands with comments

```
# First, switch to the destination branch
git switch main

# Integrate the feature branch into main
git merge feature/student-search

# Inspect the resulting history
git log --oneline --graph --decorate --all
```

What to expect / verify

If main has not diverged from the feature branch, Git may perform a fast-forward by moving the main pointer forward. If both branches have new commits, Git may create a merge commit.

Common mistakes and cautions

- Do not run `git merge main` while on the feature branch unless you intentionally want to merge main into the feature branch.
- A local merge does not automatically push to GitHub.

Remember: Switch to the branch that should receive the changes before running merge.

Lesson 15 — Fast-Forward and Three-Way Merge

Concept

A fast-forward merge is possible when the destination branch is an ancestor of the source branch. Git can move the destination pointer forward. A three-way merge uses the common ancestor and both branch tips to combine divergent histories.

Why it matters

Knowing the merge type helps you understand why Git creates a merge commit sometimes but not always.

Example scenario

Your feature branch adds two commits while `main` remains unchanged: fast-forward is possible. If a teammate also commits to main, the histories diverge and a three-way merge may be needed.

Commands with comments

```
# Show the branch graph before merging
git log --oneline --graph --all

# Merge the feature into the current branch
git merge feature/student-search

# Show the graph after merging
git log --oneline --graph --decorate --all
```

What to expect / verify

Fast-forward usually creates no new merge commit. A three-way merge may create a commit with two parents.

Common mistakes and cautions

- Do not judge whether a merge succeeded only by the last printed line; inspect `git status` and the history graph.
- Teams may use `--no-ff` to preserve an explicit merge commit for feature branches, depending on policy.

Remember: Both merge types integrate changes; they differ in how the history is recorded.

Lesson 16 — Resolve Merge Conflicts

Concept

A conflict occurs when Git cannot automatically combine changes, often because two branches changed the same lines in incompatible ways.

Why it matters

Conflicts are a normal part of collaboration. The developer must decide what the final code should say.

Example scenario

One branch changes a README sentence to mention registration; another changes the same sentence to mention search.

Commands with comments

```
# Attempt the merge from the intended destination branch
git switch main
git merge feature/student-search

# If Git reports conflicts, inspect which files are conflicted
git status

# Open each conflicted file and resolve markers:
# <<<<<<< HEAD
# current branch version
# =======
# incoming branch version
# >>>>>>> feature/student-search

# After editing the file into the correct final version:
git add README.md

# Complete the merge commit if Git requires it
git commit

# Verify that the repository is clean and history looks right
git status
git log --oneline --graph --all
```

What to expect / verify

Remove all conflict markers and preserve the intended combined result. Stage the resolved file; then complete the merge as Git directs.

Common mistakes and cautions

- Do not blindly choose one side without understanding the code.
- `git merge --abort` can abort an in-progress merge when appropriate, but inspect and protect any pre-existing work first.

Remember: Conflict resolution is a code decision, not merely a command-line task.

PART 4 — COLLABORATION ON GITHUB

Lesson 17 — Pull Requests: Purpose and Vocabulary

Concept

A Pull Request (PR) is a GitHub collaboration request to review and integrate changes from one branch into another. It is not itself a Git command.

Why it matters

PRs support discussion, code review, automated checks, and approval before changes enter a protected or stable branch.

Example scenario

You push `feature/student-search` to GitHub and propose merging it into `main`.

What to expect / verify

A PR compares a base branch (the destination, such as main) with a compare/head branch (the branch containing proposed changes). Pushing a branch does not automatically create a PR.

Common mistakes and cautions

- Do not confuse `git push` with a Pull Request. Push uploads commits; a PR asks for review and integration.

Remember: Base is where changes go; compare is where proposed changes come from.

Lesson 18 — Create, Review, and Merge a Pull Request

Concept

The standard PR workflow is: push a feature branch, open a PR, choose base and compare branches, review the diff, resolve feedback, pass required checks, and merge.

Why it matters

It creates a review checkpoint before code enters the shared branch.

Example scenario

You want your student search feature reviewed before it becomes part of main.

Commands with comments

```
# Create a feature branch and make a commit (example)
git switch -c feature/student-search
git add README.md
git commit -m "Document student search feature"

# Push the branch to GitHub
git push -u origin feature/student-search

# Then in GitHub's web interface:
# 1. Open the repository.
# 2. Choose Pull requests -> New pull request.
# 3. Base: main
# 4. Compare: feature/student-search
# 5. Review Files changed and add a useful title/description.
# 6. Select Create pull request.
# 7. Address feedback and required checks.
# 8. Merge only when approved and ready.
```

What to expect / verify

After merging on GitHub, update local main with `git switch main` followed by `git pull origin main`.

Common mistakes and cautions

- Check the base and compare branches carefully.
- A PR may require approvals, tests, or conflict resolution before GitHub allows merging.
- Do not merge unreviewed code into a production branch unless that is the agreed policy.

Remember: A PR is the review process around a proposed merge.

Lesson 19 — Team Collaboration, Reviews, and Branch Protection

Concept

Teams use review rules, automated checks, issue references, and branch protection to reduce accidental or unreviewed changes.

Why it matters

Shared projects need consistent standards, not just the ability to push code.

Example scenario

Your team requires one approval and passing tests before a PR can merge into main.

What to expect / verify

Typical workflow: create issue -> create branch -> commit focused changes -> push -> open PR -> review -> run checks -> merge -> delete the feature branch if appropriate.

Common mistakes and cautions

- A branch protection rule may prevent direct pushes or merging until required checks pass. Follow the repository's rules rather than trying to bypass them.

Remember: GitHub settings enforce team process; Git commands alone do not provide the whole review workflow.

Lesson 20 — Forks, Issues, and Contribution Workflow

Concept

A fork is a separate GitHub repository under another account. Issues track bugs, feature requests, and tasks. Contributions often flow from a fork or feature branch through a Pull Request.

Why it matters

This workflow lets contributors propose changes without needing direct write access to the original repository.

Example scenario

You find a bug in an open-source Student Management project and want to submit a fix.

Commands with comments

```
# Clone your fork
git clone https://github.com/YOUR-USERNAME/PROJECT.git
cd PROJECT

# Add the original project as an upstream remote
git remote add upstream https://github.com/ORIGINAL-OWNER/PROJECT.git

# Fetch the original project's latest updates
git fetch upstream

# Create a branch for the fix
git switch -c fix/student-validation

# After editing and testing:
git add .
git commit -m "Fix student validation"
git push -u origin fix/student-validation

# Open a Pull Request from your fork/branch to the original project.
```

What to expect / verify

Your fork is your own remote (`origin`); the original project is commonly named `upstream`.

Common mistakes and cautions

- Always check contribution guidelines and license requirements.
- Do not stage everything blindly if local files include secrets or generated output.

Remember: Issues describe work; branches contain code changes; PRs propose those changes for review.

PART 5 — UNDOING, SAVING, AND HISTORY TOOLS

Lesson 21 — Undoing Changes: restore, revert, reset, amend

Concept

Undo commands are not interchangeable. `restore` changes file contents; `revert` creates a new commit reversing an earlier commit; `reset` moves the current branch pointer and changes staging/working files depending on mode; `commit --amend` replaces the latest commit.

Why it matters

Choosing the wrong undo command can lose work or rewrite shared history. First determine whether the change is unstaged, staged, committed locally, or already pushed.

Example scenario

StudentController.cs has a bad uncommitted edit; alternatively, a faulty registration commit is already on GitHub. These are different situations and need different approaches.

Commands with comments

```
# Inspect first; make this a habit before undoing work
git status
git diff
git diff --staged
git log --oneline -5

# A. Discard only unstaged edits to a tracked file.
# WARNING: edits to this file can be lost.
git restore StudentController.cs

# B. Unstage a file while keeping its working edits.
git restore --staged StudentController.cs

# C. Reverse a commit by creating a new commit.
# Replace COMMIT_ID with the real commit ID.
git revert COMMIT_ID

# D. Move current branch back one commit, keep changes staged.
git reset --soft HEAD~1

# E. Move current branch back one commit, keep changes unstaged.
git reset --mixed HEAD~1

# F. DANGEROUS: reset branch, staging, and tracked files.
# This can discard uncommitted changes to tracked files.
# Use only after inspecting and backing up important work.
git reset --hard HEAD~1

# G. Replace the latest commit, e.g. to correct its message.
# Rewrites the latest commit and changes its ID.
git commit --amend -m "Add student registration endpoint"
```

What to expect / verify

Restore does not move the branch pointer. Revert adds a commit. Reset creates no commit and moves the pointer. Amend replaces the latest commit.

Common mistakes and cautions

- Never recommend or run `git reset --hard` without explaining the risk of losing uncommitted tracked-file changes.
- Prefer revert for commits already shared with teammates.
- Reset and amend can rewrite history; coordinate before changing published commits.
- A reset commit may remain recoverable through reflog temporarily, but do not rely on that as your only backup.

Remember: Restore = file contents; revert = new reversing commit; reset = move branch pointer; amend = replace latest commit.

Lesson 22 — Git Stash

Concept

`git stash` shelves uncommitted tracked-file changes so you can temporarily work on something else without creating a normal commit.

Why it matters

You may need to switch from unfinished registration work to an urgent login bug fix.

Example scenario

```
# Save tracked modifications and deletions with a descriptive message
git stash push -m "Student registration in progress"

# Include untracked files as well
git stash push -u -m "Registration work including new files"

# List saved stashes
git stash list

# Restore the latest stash and keep it in the stash list
git stash apply

# Or restore and remove the stash entry after successful application
git stash pop

# Inspect the working tree after restoring
git status
```

Commands with comments

```
After apply, the stash remains available. After a successful pop, Git normally removes that stash
entry.
```

What to expect / verify

A normal stash does not include untracked files; use `-u` for those. Ignored files are not included unless you explicitly use `-a`.A stash is not a permanent backup. Check the target branch before applying it.

Common mistakes and cautions

- U
- s
- e
-
- a
- p
- p
- l
- y
-
- w
- h
- e
- n
-
- y
- o
- u
-
- w
- a
- n
- t
-
- t
- o
-
- k
- e
- e
- p
-
- t
- h
- e
-
- s
- a
- v
- e
- d
-

- e
- n
- t
- r
- y
-
- u
- n
- t
- i
- l
-
- y
- o
- u
-
- v
- e
- r
- i
- f
- y
-
- t
- h
- e
-
- r
- e
- s
- t
- o
- r
- a
- t
- i
- o
- n
- .

Lesson 23 — Git Tags and Releases

Concept

A tag names a specific commit, often to mark a release such as `v1.0.0`. Unlike a branch, a release tag normally remains attached to one particular commit.

Why it matters

When you deploy a stable Student Management API version, a tag lets you identify exactly which commit was released.

Example scenario

List existing tags
git tag

Create an annotated release tag at the current commit
git tag -a v1.0.0 -m "First stable release"

Inspect the tagged commit and tag metadata
git show v1.0.0

Push one tag to GitHub
git push origin v1.0.0

Push all local tags (only if you intend to publish all of them)
git push origin --tags

Commands with comments

```
Tags are local until pushed. Annotated tags include a message, tagger, and date and are generally
useful for official releases.
```

What to expect / verify

Check the current commit before tagging.Do not reuse a tag name without inspecting the existing tag.`git push --tags` pushes every local tag.

Common mistakes and cautions

- A
-
- t
- a
- g
-
- i
- s
-
- a
-
- n
- a
- m
- e
- d
-
- m

- a
- r
- k
- e
- r
-
- f
- o
- r
-
- a
-
- s
- p
- e
- c
- i
- f
- i
- c
-
- v
- e
- r
- s
- i
- o
- n
- ;
-
- i
- t
-
- i
- s
-
- n
- o
- t
-
- a
-
- m
- o
- v
- i
- n

- g
-
- d
- e
- v
- e
- l
- o
- p
- m
- e
- n
- t
-
- b
- r
- a
- n
- c
- h
- .

Lesson 24 — Rebase and History Cleanup

Concept

`git rebase` replays commits onto a new base commit. It can create a more linear history, but the replayed commits receive new IDs.

Why it matters

Rebase is often used to update a feature branch with the latest main or to clean up local commits before sharing them.

Example scenario

You have a local feature branch and main has advanced. Your team allows rebasing the feature branch before opening a PR.

Commands with comments

```
# Start on your feature branch
git switch feature/student-search

# Fetch the latest remote history
git fetch origin

# Replay your feature commits on top of the latest origin/main
git rebase origin/main

# If conflicts occur, resolve files, stage them, then continue
git add StudentController.cs
git rebase --continue

# If you need to abandon the rebase, after checking the situation:
git rebase --abort
```

What to expect / verify

Rebase creates a linear-looking history by replaying commits. Conflicts may require repeated resolution.

Common mistakes and cautions

- Do not rebase commits that other people are already using unless the team explicitly coordinates it; rebase rewrites history.
- If a rebase stops, read Git's instructions and resolve conflicts before continuing.

Remember: Rebase changes the base of commits; merge combines histories without replaying existing commits.

Lesson 25 — Cherry-pick, Reflog, and Troubleshooting

Concept

`git cherry-pick` applies the change introduced by a selected commit onto the current branch as a new commit. `git reflog` records recent movements of local references and can help find a commit after a mistaken reset.

Why it matters

These tools are useful when you need one specific fix from another branch or need to recover from a local history mistake.

Example scenario

You fixed a validation bug on a separate branch and want only that fix on a release branch.

Commands with comments

```
# Inspect commit IDs and identify the specific fix
git log --oneline --all

# Switch to the branch that should receive the fix
git switch release/1.0

# Apply the selected commit as a new commit on this branch
git cherry-pick COMMIT_ID

# Inspect recent movements of HEAD and branch references
git reflog

# If you find the needed commit, inspect it before recovery
git show COMMIT_ID
```

What to expect / verify

Cherry-pick creates a new commit containing the selected change on the current branch. Reflog is local and entries are retained for a limited time according to Git's settings.

Common mistakes and cautions

- Cherry-pick can conflict; resolve and continue according to Git's instructions.
- Reflog is a recovery aid, not a substitute for backups. Inspect the target carefully before resetting or branching from it.

Remember: Cherry-pick selects a change; reflog helps locate where local references recently pointed.

PART 6 — PRACTICAL WORKFLOW AND CHEAT SHEET

A. A typical daily feature workflow

```
# 1. Check current state
git status

# 2. Update local main
git switch main
git pull origin main

# 3. Create a focused feature branch
git switch -c feature/student-search

# 4. Edit code, then inspect the changes
git status
git diff

# 5. Stage only intended files
git add StudentController.cs StudentService.cs

# 6. Inspect staged changes
git diff --staged

# 7. Commit a focused unit of work
git commit -m "Add student search"

# 8. Publish the branch
git push -u origin feature/student-search

# 9. Open a Pull Request on GitHub and request review
# 10. After the PR is merged, update local main
git switch main
git pull origin main
```

B. Useful commands at a glance

Goal	Command	Meaning
Check repository state	`git status`	See modified, staged, and untracked files
See unstaged changes	`git diff`	Compare working files with staging
See staged changes	`git diff --staged`	Compare staging with latest commit
Recent history	`git log --oneline -5`	Show recent commits
Branch list	`git branch`	List local branches
Switch branch	`git switch main`	Move to existing branch
Create and switch	`git switch -c feature/name`	Create branch and check it out
Download remote updates	`git fetch origin`	Update remote-tracking references
Upload commits	`git push`	Send commits to remote
Integrate remote changes	`git pull`	Fetch then integrate
Create stash	`git stash push -m "message"`	Shelf tracked changes
Inspect tags	`git tag`	List local tags

View graph	`git log --oneline --graph --all`	See branching and merge history

C. Safety checklist before undoing or merging

- Run git status.
- Review git diff and git diff --staged.
- Confirm the current branch with git branch.
- Inspect recent history with git log --oneline --graph --all.
- Identify whether the commit has already been pushed or shared.
- Save valuable uncommitted work before destructive operations.
- Never run git reset --hard without understanding that tracked uncommitted changes may be lost.
- Prefer git revert to undo a shared commit.
- After the operation, run git status and verify the files and history.

D. Troubleshooting common messages

Message / situation	Likely meaning	Safe first step
remote origin already exists	A remote named origin is already configured.	Run `git remote -v`; change its URL only if needed.
Push rejected / fetch first	Remote has commits you do not have locally.	Run `git fetch origin`, inspect logs, then integrate carefully.
Your local changes would be overwritten	Switching branches would overwrite working-tree edits.	Run `git status`; commit or stash intended work before switching.
Merge conflict	Git cannot combine some edits automatically.	Run `git status`, edit conflicted files, remove markers, stage resolutions.
No upstream branch	The local branch has no configured tracking branch.	Use `git push -u origin branch-name` if that is the intended remote.
Detached HEAD	HEAD points directly to a commit, not a local branch.	If you need to keep work, create a branch before switching away.

E. Final mental model

- Git tracks snapshots and relationships between commits.
- The working directory is where you edit files.
- The staging area selects what will go into the next commit.
- A branch is a movable pointer to a commit.
- A remote is another repository location; origin is a common remote name.
- Push uploads commits; fetch downloads remote information; pull fetches and integrates.
- Merge integrates histories; rebase replays commits onto another base.
- A Pull Request is a GitHub review and integration workflow.
- Restore changes file contents; revert creates a reversing commit; reset moves the branch pointer; amend replaces the latest commit.
- Inspect before acting, especially before commands that discard work or rewrite shared history.

Practice Plan

1. Create a disposable practice repository and make three small commits.
2. Practise status, diff, staging, and commit before trying branches.
3. Create a feature branch and merge it back into main.
4. Create a deliberate README conflict and resolve it.
5. Push a feature branch to a test GitHub repository and create a Pull Request.
6. Practise restore, revert, and reset in a disposable repository only.
7. Create a stash, apply it, and verify the restored changes.
8. Create and push an annotated tag.
9. Practise rebase and cherry-pick only after you understand commits and branches.

Final safety rule: When you are unsure what Git will change, stop and run git status, git diff, git diff --staged, and git log --oneline --graph --all. Do not use destructive commands as a guess.